Lecture 10. Functions

Writing functions, conditional execution, arguments, return values, environments

Contents

10.1 When to write a function **2**

10.2 Functions are for humans and computers **2**

10.3 Conditional execution **3**

10.3.1 Conditions . . . 3

10.3.2 Multiple conditions . . . 3

10.3.3 Code style . . . 4

10.4 Function arguments **4**

10.4.1 Checking values . . . 4

10.4.2 Dot-dot-dot . . . 5

10.5 Return values **5**

10.6 Environment **5**

10.7 Selected exercises **6**

10.1 When to write a function

Motivation. Functions remove copy-and-paste bugs and name an idea.

Rescale each column to $[0, 1]$:

```
df <- tibble::tibble(a = rnorm(10), b = rnorm(10), c = rnorm(10), d = rnorm(10))
df$a <- (df$a - min(df$a, na.rm = TRUE)) /
  (max(df$a, na.rm = TRUE) - min(df$a, na.rm = TRUE))
df$b <- (df$b - min(df$b, na.rm = TRUE)) /
  (max(df$b, na.rm = TRUE) - min(df$a, na.rm = TRUE))
# ... and the same for c and d
```

The `b` line has a hidden error (`min(df$a)`). Rule: copied more than twice, write a function (DRY).

How. Count inputs (one: call it `x`); remove duplication (`range()` once instead of three times):

```
rescale01 <- function(x) {
  rng <- range(x, na.rm = TRUE)
  (x - rng[1]) / (rng[2] - rng[1])
}
rescale01(c(-10, 0, 10))
```

```
## [1] 0.0 0.5 1.0
```

```
rescale01(c(1, 2, 3, NA, 5))
```

```
## [1] 0.00 0.25 0.50   NA 1.00
```

A **name**, **arguments** in `function()`, a **body** in braces; start from working code. Each column is now `df$a <- rescale01(df$a)` (Lecture 12 removes this too).

Changes happen in one place: infinite values make everything `NaN`, and one edit fixes every use:

```
x <- c(1:10, Inf)
rescale01 <- function(x) {
  rng <- range(x, na.rm = TRUE, finite = TRUE)
  (x - rng[1]) / (rng[2] - rng[1])
}
rescale01(x)
```

```
##  [1] 0.0000000 0.1111111 0.2222222 0.3333333 0.4444444 0.5555556 0.6666667 0.7777778
##  [9] 0.8888889 1.0000000       Inf
```

10.2 Functions are for humans and computers

R does not care about names or comments, but readers do.

- **Names**: short and descriptive; verbs for functions (`impute_missing()`), nouns for arguments; not `f()` or `my_awesome_function()`.
- One style, `snake_case` or `camelCase`, not `col_mins()` with `rowMaxes()`; a common prefix for a family (`input_select()`, `input_text()`).
- Never overwrite `T <- FALSE`, `c <- 10`, `mean <- function(x) sum(x)`.
- **Comments** explain *why*; the code shows what and how. `# Load data ---` lines make sections.

10.3 Conditional execution

Motivation. Branches for special cases and choices of method.

```
if (condition) {
  # code executed when condition is TRUE
} else {
  # code executed when condition is FALSE
}
```

```
has_name <- function(x) {
  nms <- names(x)
  if (is.null(nms)) {
    rep(FALSE, length(x))
  } else {
    !is.na(nms) & nms != ""
  }
}
has_name(c(a = 1, 2, b = 3))
```

```
## [1]  TRUE FALSE  TRUE
```

A function returns its last value.

10.3.1 Conditions

One `TRUE` or `FALSE`; a longer vector (formerly a warning) or `NA` is an error:

```
if (c(TRUE, FALSE)) {}
```

```
## Error in if (c(TRUE, FALSE)) {: the condition has length > 1
```

```
if (NA) {}
```

```
## Error in if (NA) {: missing value where TRUE/FALSE needed
```

Combine with short-circuit `||`, `&&`, not vectorised `|`, `&`; reduce vectors with `any()`, `all()`. `==` is vectorised and gives `NA`; `identical()` returns one value but distinguishes types (`identical(0L, 0)` is `FALSE`); for reals `dplyr::near()`, as `sqrt(2)^2 == 2` is `FALSE`.

10.3.2 Multiple conditions

Chain `if (this) {...} else if (that) {...} else {...}`; `switch()` selects by name or position:

```
calc <- function(x, y, op) {
  switch(op,
    plus = x + y,
    minus = x - y,
    times = x * y,
    divide = x / y,
    stop("Unknown op!")
  )
}
calc(6, 3, "divide")
```

```
## [1] 2
```

`cut()` replaces long `if` chains for continuous variables.

10.3.3 Code style

An opening `{` ends a line, never alone; a closing `}` gets its own line unless `else` follows; indent two spaces. Short: `x <- if (y < 20) "Too low" else "Too high"`. `else` on a new line after `}` is a top-level syntax error (the `if` is complete).

10.4 Function arguments

Motivation. The interface: data versus details, sensible defaults, loud failures.

Data arguments first; **detail** arguments last, usually with **defaults**:

```
mean_ci <- function(x, conf = 0.95) {
  se <- sd(x) / sqrt(length(x))
  alpha <- 1 - conf
  mean(x) + se * qnorm(c(alpha / 2, 1 - alpha / 2))
}
x <- runif(100)
mean_ci(x)
```

```
## [1] 0.4194814 0.5276171
```

```
mean_ci(x, conf = 0.99)
```

```
## [1] 0.4024921 0.5446064
```

Default = the most common value (except `na.rm = FALSE`: ignoring missing values must be deliberate). Call with unnamed data and fully named details: `mean(1:10, na.rm = TRUE)`, not `mean(, TRUE, x = c(1:10, NA))`; spaces around `=` and after commas.

Common names: `x`, `y`, `z` (vectors), `w` (weights), `df`, `i`, `j` (indices), `n` (length, rows), `p` (columns); follow conventions like `na.rm`.

10.4.1 Checking values

Recycling hides mistakes:

```
wt_mean <- function(x, w) sum(x * w) / sum(w)
wt_mean(1:6, 1:3)
```

```
## [1] 7.666667
```

Weights recycled to length 6, no warning (6 is a multiple of 3). Check with `stop()` or `stopifnot()` (each argument must be `TRUE`):

```
wt_mean <- function(x, w, na.rm = FALSE) {
  stopifnot(is.logical(na.rm), length(na.rm) == 1)
  stopifnot(length(x) == length(w))
  if (na.rm) {
    miss <- is.na(x) | is.na(w)
    x <- x[!miss]
    w <- w[!miss]
  }
  sum(w * x) / sum(w)
}
wt_mean(1:6, 6:1, na.rm = "foo")
```

```
## Error in wt_mean(1:6, 6:1, na.rm = "foo"): is.logical(na.rm) is not TRUE
```

Checks trade time for robustness.

10.4.2 Dot-dot-dot

`...` captures all unmatched arguments (as in `sum()`) and passes them on:

```
commas <- function(...) stringr::str_c(..., collapse = ", ")
commas(letters[1:10])
```

```
## [1] "a, b, c, d, e, f, g, h, i, j"
```

Misspellings are swallowed: `sum(x, na.mr = TRUE)` adds `TRUE`, i.e. 1.

10.5 Return values

`return()` exits early, for simple special cases:

```
complicated_function <- function(x, y, z) {
  if (length(x) == 0 || length(y) == 0) {
    return(0)
  }
  # complicated code here
}
```

Pipeable functions. *Transformations* return a modified first argument; *side-effect* functions return it *invisibly*:

```
show_missings <- function(df) {
  n <- sum(is.na(df))
  cat("Missing values: ", n, "\n", sep = "")
  invisible(df)
}
mtcars %>%
  show_missings() %>%
  mutate(mpg = ifelse(mpg < 20, NA, mpg)) %>%
  show_missings()
```

```
## Missing values: 0
## Missing values: 18
```

10.6 Environment

Motivation. Where R looks up variables that are not arguments.

```
f <- function(x) {
  x + y
}
y <- 100
f(10)
```

```
## [1] 110
```

```
y <- 1000
f(10)
```

```
## [1] 1010
```

Lexical scoping: a name not found inside is looked up where the function was *defined* (here global), so the result depends on a global variable: avoid. The same rules let R override even `+` or `(`.

Defined, not called [Supplement]

```
make_adder <- function(y) function(x) x + y
add2 <- make_adder(2)
y <- 1000
add2(10)
```

```
## [1] 12
```

`add2` keeps `y = 2` from its creating environment (a *closure*); the global `y` is irrelevant.

10.7 Selected exercises

Motivation. Interfaces, scalar versus vector conditions, defaults, and lookup time.

Exercise 10.1 (R4DS Exercise 19.2.1.1)

Why is `TRUE` not an argument of `rescale01()`? What would happen if `x` contained a single missing value and `na.rm` were `FALSE`?

```
rescale01_strict <- function(x) {
  rng <- range(x, na.rm = FALSE)
  (x - rng[1]) / (rng[2] - rng[1])
}
rescale01_strict(c(1, 2, NA, 4))
```

```
## [1] NA NA NA NA
```

Solution.

Step 1. Arguments are what callers may want to change. Mapping observed values to $[0, 1]$ needs the range without `NA`s; `na.rm = FALSE` has no sensible use, so it is fixed in the body.

Step 2. With `na.rm = FALSE` one `NA` gives `range()` = `c(NA, NA)`, so every entry, observed or not, is `NA`: one missing value destroys the column. ■

Exercise 10.2 (R4DS Exercises 19.4.4.1 and 19.4.4.5)

(a) What is the difference between `if` and `ifelse()`?
(b) What happens if `switch()` is used with numeric values?

```
ifelse(c(TRUE, FALSE, NA), "yes", "no")
```

```
## [1] "yes" "no"  NA
```

```
if (c(TRUE, FALSE)) "yes" else "no"
```

```
## Error in if (c(TRUE, FALSE)) "yes" else "no": the condition has length > 1
```

```
switch(2, "a", "b", "c")
```

```
## [1] "b"
```

```
is.null(switch(4, "a", "b", "c"))
```

```
## [1] TRUE
```

Solution.

(a) `if` is control flow: one `TRUE`/`FALSE` condition selects one of two blocks. `ifelse(test, yes, no)` is vectorised: length of `test`, elements from `yes` or `no`, `NA` where `test` is `NA`. `if` chooses what

to do; `ifelse()` computes a column.

(b) With a number n, `switch()` ignores names and returns the nth remaining argument; beyond their number, invisible `NULL` (hence `is.null()`). ■

Exercise 10.3 (R4DS Exercise 19.5.5.3)

The default value of the `method` argument of `cor()` is `c("pearson", "kendall", "spearman")`. What does that mean, and which value is used by default?

```
centre <- function(x, type = c("mean", "median")) {
  type <- match.arg(type)
  switch(type, mean = mean(x), median = median(x))
}
centre(c(1, 2, 10))
```

```
## [1] 4.333333
```

```
centre(c(1, 2, 10), "median")
```

```
## [1] 2
```

```
centre(c(1, 2, 10), "mode")
```

```
## Error in match.arg(type): 'arg' should be one of "mean", "median"
```

Solution. The vector lists the allowed values for `match.arg()`. Omitted, it gets the whole vector and returns the first element: `"pearson"`. Supplied, it must be listed (partial matching allowed), else an error. So `centre()` defaults to the mean, allows the median, rejects `"mode"`; the signature documents the choices. ■

Exercise 10.4 (when is a free variable looked up)

Predict the value of `f()`, and explain it with lexical scoping.

```
f <- function() {
  x <- 1
  g <- function() x
  x <- 2
  g()
}
f()
```

```
## [1] 2
```

Solution [Supplement].

Step 1. `g` has no local `x`; it is looked up in `g`'s defining environment, `f`'s execution environment.

Step 2. Scoping fixes *where*, not *when*: at the call `g()`, that `x` is already 2, so `g()` returns 2.

Step 3. Consistent with `add2`: nothing changes `y` in its defining environment; the global `y` is elsewhere. ■